State of the Practice for Open-Source Neural Network Software

Department of Computing and Software
McMaster University
Hamilton, Ontario, Canada

Abstract

Open-source neural network (NN) software plays an important role in modern AI research and development. Assessing the software-engineering practices of NN packages, however, is challenging because relevant evidence is distributed across public artifacts such as documentation, tests, issue trackers, releases, and continuous integration (CI) configurations.

This paper presents DomainX, a tool-supported workflow for conducting a state-of-practice assessment of open-source NN software based on a published methodology. DomainX supports evidence capture, scoring through a measurement template spanning nine software-quality attributes, incorporation of repository-based measures, and overall comparison using the Analytic Hierarchy Process (AHP).

We apply this workflow to a curated shortlist of 28 open-source NN software packages and produce a traceable dataset containing template responses, repository-based measures, and package-level references (project website and repository URLs). Keras achieved the highest overall score, followed closely by TensorFlow, Hugging Face Transformers, and PyTorch. The leading packages are tightly clustered, with the overall ranking shaped by a broad mix of categories, including installability, correctness and verifiability, surface reliability, surface usability, maintainability, and reusability. A companion repository provides the DomainX tool and associated assessment artifacts to support review and reuse.

Keywords: state of practice; research software; neural network libraries; software quality; measurement template; Analytic Hierarchy Process (AHP); traceability; reproducibility; repository mining.

1. Introduction

Research and engineering outcomes increasingly depend on software, and the quality of that software affects reliability, maintainability, and reproducibility. In domains where open-source libraries are widely reused, understanding current development practice is an important step toward improving it. A *state-of-practice* assessment provides a structured way to characterize what developers commonly do in a domain by evaluating representative software packages against explicit software-quality criteria using traceable evidence.

Conducting such assessments rigorously is challenging in practice. Evidence relevant to software quality is distributed across many public artifacts, including documentation, tutorials, repository history, tests, issue trackers, releases, and continuous integration (CI) configurations. This makes the assessment process time-consuming and creates challenges for auditability, maintenance, and reproducibility as repositories evolve over time.

This paper presents a tool-supported state-of-practice assessment for *neural network (NN) software*. The target audience is developers and maintainers of NN software; accordingly,

the scope is restricted to NN packages rather than general-purpose NN-supporting libraries from adjacent communities. The assessment workflow follows a published state-of-practice methodology based on (i) a measurement template organized by software-quality attributes, (ii) repository-based measures extracted from public repositories, and (iii) an Analytic Hierarchy Process (AHP) ranking procedure to combine evidence into transparent comparisons.

The contributions of this work are:

- A curated shortlist of 28 open-source NN software packages evaluated using a consistent, evidence-based procedure (Appendix A).
- A traceable assessment dataset recording measurement-template responses, repository-based measures, and package-level references (project website and repository URLs) for auditability and reuse.
- DomainX, a tool that supports evidence capture, scoring, ranking computations, and export of results to reduce manual effort and improve traceability (`https://github.com/thaafei/DomainX`).

The remainder of this paper defines key concepts and summarizes background (Section 2), describes the assessment methodology and tool support (Section 3), and reports results and observations from applying the approach to the shortlisted packages (Section 4). The paper concludes with discussion, recommendations, and limitations (Sections 5–7).

2. Background and Definitions

In this paper, *research software* refers to software used to support scientific or engineering work such as data processing, model training or inference, experimentation, and analysis. A *domain* is a coherent problem area characterized by shared goals and typical workflows; in a state-of-practice assessment, the domain determines which software packages are relevant and what evidence is meaningful for evaluation. A *library* (or package) is a reusable software component intended to be integrated into other programs. We use *evidence* to mean publicly available artifacts that indicate software-development practice, including documentation and tutorials, repository history (commits and releases), issue trackers and pull requests, tests, and continuous integration (CI) configurations.

The *state of the art* describes the most advanced methods and recommended best practices reported in research and practitioner literature. The *state of practice* describes what is commonly done in real projects: the tools used, the artifacts produced, and the quality-related practices that are routinely applied. Gaps between state of the art and state of practice are common, particularly in research settings where incentives and timelines may prioritize functionality and novelty over long-term maintainability, transparency, or reproducibility. A state-of-practice assessment aims to characterize this reality using explicit criteria and traceable evidence, enabling systematic comparison and targeted recommendations.

A published state-of-practice methodology guides our assessment by combining (i) a measurement template grounded in software-quality attributes with (ii) evidence extracted from public artifacts and repository-based measures, enabling comparisons that are intended to be reproducible and auditable Smith et al. (October 2021).

To produce an overall comparison across multiple criteria, we use the Analytic Hierarchy Process (AHP), a multi-criteria decision-making method that derives weights from pairwise comparisons of criteria and aggregates scores accordingly. AHP is well suited to this setting

because it makes weighting assumptions explicit and supports transparent trade-offs among software-quality attributes. In this work, AHP is used to compute an evidence-based ranking and to support sensitivity analysis on how rankings change under different weighting choices.

3. Methods

3.1. Domain framing and scope

Neural network (NN) software is defined here as open-source libraries whose primary purpose is to enable the development, training, deployment, analysis, or direct use of neural network models. The target audience for this assessment is developers and maintainers of NN software. Accordingly, general-purpose NN-supporting packages (e.g., numerical computing, data manipulation, or plotting libraries) are out of scope because they belong to different developer communities and would confound interpretation of NN software development practice. Packages are included when they have a publicly accessible repository and sufficient publicly available artifacts (e.g., documentation and repository history) to support evidence-based evaluation.

3.2. Candidate identification and filtering

Candidate packages were identified by aggregating package names from authoritative sources and normalizing duplicates and naming variants. Packages that were primarily NN-supporting rather than neural network software were removed to keep the assessment aligned with the intended audience of neural network software developers and maintainers.

Packages that appeared only once across the authoritative sources were not automatically excluded. In cases where the domain expert advised that such packages were still relevant to the domain, they were retained to improve completeness of coverage.

After the initial shortlisting stage, 31 candidate packages remained. A further round of shortlist validation produced the final assessed set of 28 packages. The final shortlist and the packages excluded at the later stage, together with brief exclusionary reasons, are provided in Appendix Appendix A.

3.3. Evidence sources and data collection

Evidence was derived from publicly available artifacts associated with each package, including documentation (e.g., README and installation guides), tutorials/examples, repository history, releases, issue trackers and pull requests, tests, and continuous integration (CI) configuration when available. Repository-based measures were extracted using automated scripts/tools. Some measurement-template questions required manual judgment (e.g., installation and usage guidance, presence of tests, or CI configuration). Responses were recorded directly in the measurement template, and each package entry includes a project website URL and a source-code repository URL that can be used to verify the recorded responses against the corresponding public artifacts.

3.4. Measurement template and scoring

The assessment uses a published measurement template organized by software-quality attributes. The qualities assessed are: installability; correctness and verifiability; surface reliability; surface robustness; surface usability; maintainability; reusability; surface understandability; and visibility/transparency. For each package, publicly available evidence was used to answer template questions and derive per-quality scores according to the template's scoring

rules. Scores were normalized and aggregated to support cross-package comparison. When evidence was missing or ambiguous, responses were recorded conservatively and flagged for traceability.

3.5. Repository-based measures

Repository-based measures complement the measurement template by capturing observable project characteristics and development activity from public repositories. Using the same repository-measure framework defined in the methodology paper, the collected raw data includes GitHub-derived measures such as stars, forks, watchers, open and closed pull requests, and open and closed issues, together with repository statistics such as number of developers, programming languages, file counts, lines of code, comment lines, blank lines, binary files, total commits, and commit activity over recent yearly and monthly intervals Smith et al. (October 2021). The methodology identifies `git_stats` and `scc` as the tools used to collect repository statistics, with GitHub repository data used when the project is hosted on GitHub Smith et al. (October 2021).

In addition to raw measures, the methodology defines a small set of processed measures derived from the raw data. These include project status (alive or dead), percentage of issues closed, and percentage of code that is comments Smith et al. (October 2021). Together, these measures provide additional context for interpreting the quality scores recorded in the measurement template.

3.6. Ranking method (AHP)

To produce an overall comparison across multiple quality criteria, this work uses the Analytic Hierarchy Process (AHP), following the methodology paper Smith et al. (October 2021). AHP is a multi-criteria decision-making technique based on pairwise comparisons. In this context, it is used to compare software packages using the quality scores derived from the measurement template. The pairwise comparisons are used to construct matrices that yield an overall score for each package under the selected criteria weights Smith et al. (October 2021).

As emphasized in the methodology, the ranking is not meant to identify a single universally "best" package. Instead, it is intended to provide insight into the top-performing set of packages and to support interpretation of domain-wide patterns Smith et al. (October 2021). The methodology also includes sensitivity analysis so that the stability of the ranking can be examined under uncertainty or variation in the quality scores and weighting choices Smith et al. (October 2021).

3.7. Tool support: DomainX

DomainX supports a tool-based workflow for conducting the assessment and managing the resulting dataset. It provides an interactive interface for recording measurement-template responses and repository-based measures across the shortlisted packages. DomainX also integrates the Analytic Hierarchy Process (AHP) computation used to aggregate quality-level scores into an overall comparison and visualizes the resulting ranking. In addition, the tool supports export of assessment artifacts for reporting and review. The DomainX implementation is available at `https://github.com/thaafei/DomainX`.

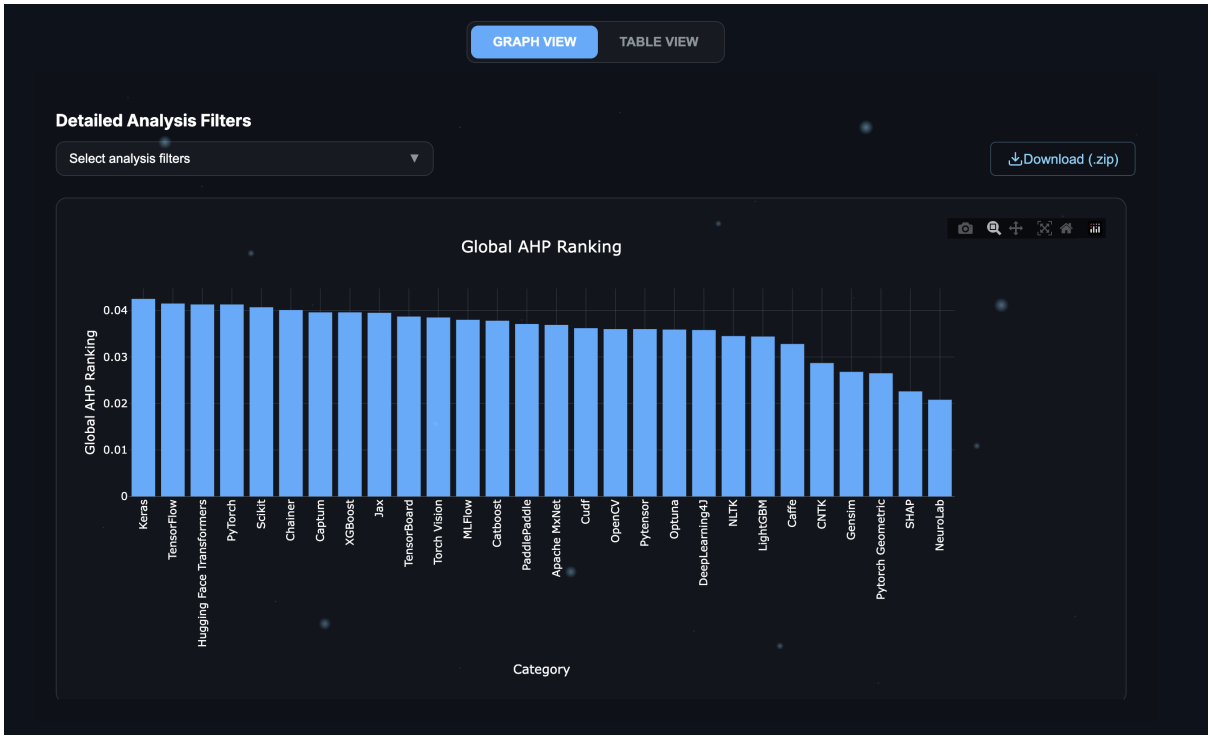


Figure 1: DomainX graph view illustrating the overall AHP ranking for the assessed packages.

3.8. Environment and reproducibility

Following the methodology, installation-based assessment is performed in a clean, isolated environment so that software packages are evaluated on a more level playing field Smith et al. (October 2021). The methodology recommends using virtual machines (VMs) because they provide a fresh environment for each installation attempt, reduce conflicts with pre-existing libraries, and help avoid "works-on-my-computer" effects Smith et al. (October 2021).

In the methodology, repository-based data is collected first, followed by installation and measurement-template scoring. The methodology also recommends recording environment details for each installation context, including the operating system and virtualization environment, to support repeatability Smith et al. (October 2021). In the present work, some older packages required additional environment setup because their dependencies and installation requirements were not readily compatible with a modern default environment. In these cases, containerized environments defined through Dockerfiles were used to reproduce a more suitable setup and improve consistency across installation attempts. These choices were made to improve consistency and transparency of the assessment process.

4. Results

4.1. Descriptive summary of assessed packages

The final shortlist contains 28 open-source neural network software packages selected according to the scope and filtering process described in Section 3. Based on the project-status information recorded in the assessment template, 23 packages were classified as alive and 5 were classified as dead. This mix of active and inactive projects provides important context for interpreting the assessment results, since project activity can affect the availability of documentation, testing artifacts, maintenance signals, and other observable evidence of development practice.

4.2. *Overall ranking*

Figure 2 presents the overall AHP ranking for the shortlisted packages as produced by DomainX. Keras achieved the highest overall score (4.25%), followed closely by TensorFlow (4.15%), Hugging Face Transformers (4.13%), PyTorch (4.13%), and Scikit (4.07%). At the lower end of the ranking, NeuroLab (2.08%), SHAP (2.26%), PyTorch Geometric (2.65%), Gensim (2.68%), and CNTK (2.87%) obtained the smallest overall scores.

A notable feature of the ranking is the tight clustering at the top of the shortlist. The leading packages are separated by relatively small margins, suggesting that multiple packages perform strongly across the assessed categories. At the same time, the lower-ranked packages remain meaningfully separated from the top group, particularly where weaker scores appear across several categories rather than just one.

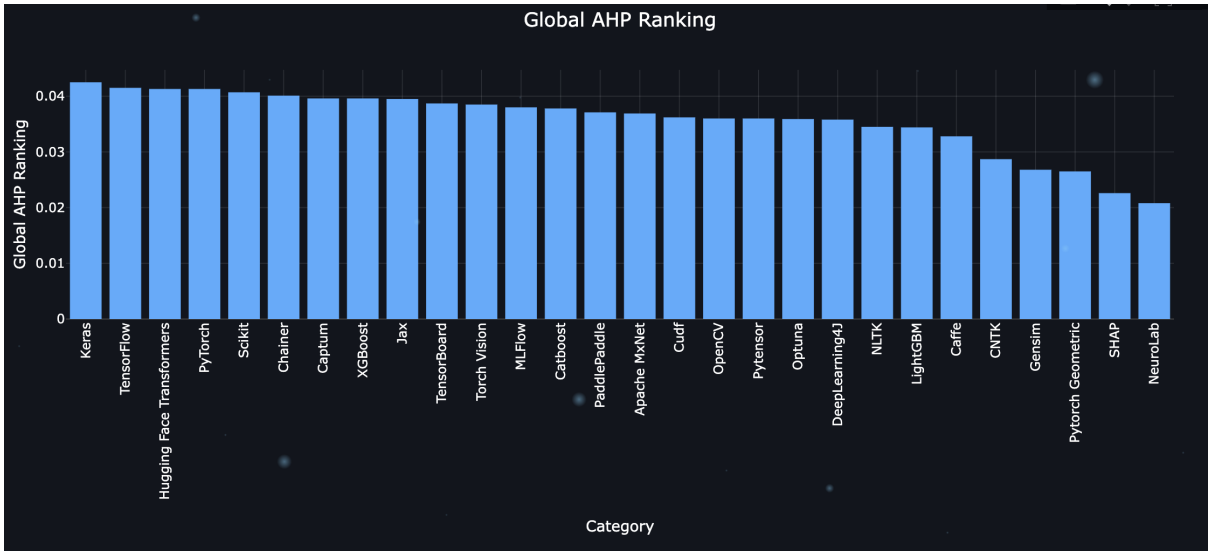


Figure 2: Overall AHP ranking for the shortlisted packages as visualized in DomainX.

Rank	Package	Overall Score
1	Keras	4.25%
2	TensorFlow	4.15%
3	Hugging Face Transformers	4.13%
4	PyTorch	4.13%
5	Scikit	4.07%
6	Chainer	4.01%
7	Captum	3.96%
8	XGBoost	3.96%
9	Jax	3.95%
10	TensorBoard	3.87%

Table 1: Top 10 packages by overall AHP score.

Table 1 lists the top 10 packages in the overall AHP ranking. Keras is the top-ranked package, but the leading group is tightly clustered, with TensorFlow, Hugging Face Transformers, and PyTorch close behind.

4.3. Interpreting the leading cluster

The leading positions are not driven by a single overwhelmingly dominant package, but rather by a small cluster of packages that perform strongly across several of the most differentiating categories. Keras, TensorFlow, Hugging Face Transformers, and PyTorch all score well overall because they combine comparatively strong values across installability, correctness and verifiability, surface usability, maintainability, and reusability.

Keras achieves the highest overall score, but its lead over TensorFlow, Hugging Face Transformers, and PyTorch is modest. This indicates that the top of the ranking is best interpreted as a competitive group rather than a single clear winner separated from all other packages by a large margin. Packages that are consistently strong across multiple categories are favored over packages that excel in only one or two dimensions.

This is important because it shows that the ranking should be read as an evidence-based comparison rather than as a universal claim that one package is best in all respects. The relatively small gaps within the leading group also suggest that the ordering at the top is sensitive to small differences across several categories, even if the same general set of high-performing packages remains near the front of the ranking.

4.4. Quality-by-quality breakdown

Figure 3 summarizes the per-quality results across the 28 shortlisted packages. The results show meaningful variation across several categories. Installability, correctness and verifiability, surface reliability, surface usability, maintainability, and reusability all contribute to the observed differences in the final ranking, while surface robustness and visibility/transparency are more stable across much of the shortlist.

Installability is an important differentiator. Several packages, including CNTK, Cudf, Keras, NLTK, OpenCV, PaddlePaddle, Scikit, TensorFlow, Torch Vision, and XGBoost, obtain the highest value in this category (5.93%), while others such as LightGBM and TensorBoard score substantially lower. Correctness and verifiability also separates packages strongly: a large group including Captum, Chainer, Gensim, Hugging Face Transformers, Jax, Keras, Optuna, PyTorch, PyTorch Geometric, Scikit, TensorBoard, and TensorFlow receives the highest value (5.80%), while several other packages score notably lower.

Additional categories also contribute visibly to the final ordering. Surface reliability distinguishes packages such as Apache MXNet, Captum, Catboost, Chainer, and TensorFlow from lower-scoring packages such as Caffe, Cudf, LightGBM, NLTK, OpenCV, and Scikit. Surface usability varies meaningfully, with strong values for packages such as Captum, CNTK, Hugging Face Transformers, Keras, Scikit, and TensorFlow, and weaker values for packages such as OpenCV, Caffe, and Cudf. Reusability and maintainability further separate stronger packages from weaker performers, while visibility/transparency is relatively stable except for a few low-scoring outliers such as NeuroLab and SHAP.

Overall, the final ordering reflects cumulative differences across several categories, with the strongest packages performing consistently well across multiple dimensions of the assessment.

Library	Installability (%)	Correctness and Verifiability (%)	Surface Reliability (%)	Surface Robustness (%)	Surface Usability (%)	Maintainability (%)	Reusability (%)	Visibility (%)	Surface Understandability (%)
Apache MxNet	3.48	2.28	4.49	3.68	1.88	3.92	4.84	3.81	2.00
Caffe	2.15	0.52	0.73	3.68	1.23	3.92	1.86	3.81	4.00
Captum	3.48	5.80	4.49	3.68	5.66	3.92	1.22	3.81	4.00
Catboost	2.15	2.28	4.49	3.68	3.13	3.92	4.84	3.81	4.00
Chainer	3.48	5.80	4.49	3.68	3.13	3.92	4.84	3.81	4.00
CNTK	5.93	2.28	4.49	3.68	5.66	1.96	4.84	3.81	4.00
Cudf	5.93	0.85	0.73	3.68	1.23	3.92	4.84	3.81	4.00
DeepLearning4J	3.48	0.65	4.49	3.68	1.23	3.92	4.84	3.81	2.00
Gensim	2.15	5.80	4.49	3.68	5.66	1.96	1.22	3.81	4.00
Hugging Face Transformers	3.48	5.80	4.49	3.68	5.66	3.92	4.84	3.81	4.00
Jax	2.15	5.80	4.49	3.68	3.13	3.92	4.84	3.81	4.00
Keras	5.93	5.80	4.49	3.68	5.66	3.92	4.84	3.81	4.00
LightGBM	0.61	1.58	0.73	3.68	5.66	3.92	1.22	3.81	4.00
MLFlow	0.78	3.53	4.49	3.68	5.66	3.92	4.84	3.81	2.00
NeuroLab	1.43	2.28	4.49	3.68	3.13	1.96	0.68	0.41	2.00
NLTK	5.93	0.85	0.73	3.68	1.23	3.92	1.22	3.81	4.00
OpenCV	5.93	0.85	0.73	3.68	0.87	3.92	4.84	3.81	4.00
Optuna	1.43	5.80	4.49	0.61	3.13	3.92	1.22	3.81	4.00
PaddlePaddle	5.93	0.85	4.49	3.68	1.23	3.92	4.84	3.81	2.00
Pytensor	1.43	2.28	4.49	3.68	3.13	3.92	1.86	3.81	4.00
PyTorch	3.48	5.80	4.49	3.68	5.66	3.92	4.84	3.81	4.00
Pytorch Geometric	3.48	5.80	1.36	3.68	3.13	1.96	4.84	3.81	4.00
Scikit	5.93	5.80	0.73	3.68	5.66	3.92	4.84	3.81	4.00
SHAP	1.43	3.53	4.49	3.68	3.13	1.96	1.22	0.60	4.00
TensorBoard	0.61	5.80	4.49	3.68	3.13	3.92	4.84	3.81	4.00
TensorFlow	5.93	5.80	4.49	3.68	5.66	3.92	4.84	3.81	2.00
Torch Vision	5.93	3.53	4.49	3.68	3.13	3.92	1.22	3.81	4.00
XGBoost	5.93	2.28	4.49	3.68	3.13	3.92	4.84	3.81	4.00

Figure 3: Heatmap summarizing per-quality scores for the 28 shortlisted neural network software packages. Darker cells indicate larger values within a given category.

5. Discussion and Recommendations

The results indicate that the overall ranking is shaped by cumulative differences across several categories rather than by a single dominant outlier. The highest-ranked packages perform consistently well across multiple dimensions of the assessment, including installability, correctness and verifiability, surface reliability, surface usability, maintainability, and reusability. This means that the leading positions are best interpreted as a competitive cluster of packages with broadly strong performance rather than a ranking dominated by one package with an exceptional score in only one category.

For users of neural network software, the ranking provides a useful evidence-based comparison, but it should not be interpreted as a universal statement that one package is always best for every purpose. Instead, the results are most useful for identifying strong candidates and for showing which quality dimensions contribute most to the observed differences across packages. For maintainers, the results highlight areas where visible software-engineering practice can be strengthened and made more consistent.

Several recommendations follow from these findings. First, maintainers should improve installability by providing clear installation instructions, dependency information, and environment guidance. Second, projects should strengthen correctness and verifiability by exposing more visible evidence of testing, validation, and related quality assurance practices. Third, maintainers should improve surface usability by providing clear tutorials, expected outputs, and accessible user-support resources. Fourth, maintainers should continue to strengthen maintainability and reusability through well-documented contribution processes, API documentation, and modular code structure. Finally, projects should preserve transparent public artifacts, since these materials directly affect the evidence available for state-of-practice assessment.

6. Threats to Validity and Limitations

This work is subject to several threats to validity. Construct validity is limited by the fact that the assessment relies on publicly visible artifacts as evidence of software-engineering practice. A package may implement strong internal practices that are not clearly reflected in its repository, documentation, or related public materials. Similarly, some categories may appear

more or less differentiated depending on the kinds of evidence that are available publicly and on how the template operationalizes those qualities.

Internal validity is affected by the need for manual judgment when answering some measurement-template questions. Although the methodology and template provide structure, interpretation of public artifacts can still introduce subjectivity. In addition, the current dataset records package-level references through project website and repository URLs rather than per-question artifact links, which limits the granularity of traceability.

External validity is limited because the assessment focuses on a specific shortlist of 28 open-source neural network software packages. The results therefore should not be generalized automatically to all neural network software or to adjacent communities using general-purpose supporting libraries. The scope was intentionally restricted to align with the target audience of neural network software developers and maintainers.

Temporal validity is also a limitation. Open-source repositories evolve over time as releases, issues, pull requests, documentation, and tests change. As a result, the assessment reflects the state of the observed artifacts at the time of data collection rather than a permanent ranking. Finally, interviews with developers or maintainers were not conducted at this stage, so the analysis is based only on publicly available evidence and repository-derived measures.

7. Conclusion and Future Work

This paper presented a state-of-practice assessment of 28 open-source neural network software packages using DomainX, a tool-supported workflow based on a published methodology. The work contributes a curated shortlist, a traceable dataset containing measurement-template responses and repository-based measures, and a tool that supports evidence capture, scoring, ranking, and export of assessment artifacts.

The results show that the highest-ranked packages form a tightly clustered leading group rather than a ranking dominated by a single extreme outlier. Keras achieved the highest overall score, followed closely by TensorFlow, Hugging Face Transformers, and PyTorch. More broadly, the ranking reflects cumulative differences across several categories, with stronger packages performing consistently well across installability, correctness and verifiability, surface reliability, surface usability, maintainability, and reusability.

Future work includes refining categories that are currently weakly differentiated, conducting sensitivity analysis to examine how the ranking changes under alternative AHP weight configurations, and extending the assessment with additional validation mechanisms such as interviews with developers or maintainers. Further work on DomainX may also improve automation, traceability granularity, and support for repeated assessments as repositories evolve over time.

Appendix A. Assessed packages and exclusions

Table A.2 lists the 28 packages included in the final assessed set used throughout this paper. Packages that were not retained in the final assessed set are listed separately in Table A.3 for traceability.

#	Package	Status
1	Apache MXNet	Dead
2	Caffe	Dead
3	Captum	Alive
4	Catboost	Alive
5	Chainer	Dead
6	CNTK	Dead
7	Cudf	Alive
8	DeepLearning4J	Alive
9	Gensim	Alive
10	Hugging Face Transformers	Alive
11	Jax	Alive
12	Keras	Alive
13	LightGBM	Alive
14	MLFlow	Alive
15	NeuroLab	Dead
16	NLTK	Alive
17	OpenCV	Alive
18	Optuna	Alive
19	PaddlePaddle	Alive
20	Pytensor	Alive
21	PyTorch	Alive
22	PyTorch Geometric	Alive
23	Scikit	Alive
24	SHAP	Alive
25	TensorBoard	Alive
26	TensorFlow	Alive
27	TorchVision	Alive
28	XGBoost	Alive

Table A.2: Final assessed set of 28 neural network software packages.

Package	Reason
Theano	Excluded from the final paper analysis because it is better characterized as a legacy tensor-expression / computational backend and historical precursor than as a current representative neural-network software package.
ONNX	Excluded from the final paper analysis because ONNX is primarily an interoperability standard / model format ecosystem rather than a core neural-network software library of the same kind as the finalized shortlisted frameworks.
NNabla	Considered during shortlist development but not retained in the final paper analysis after shortlist validation.

Table A.3: Packages not retained in the final paper analysis

Appendix B. Measurement template excerpt and scoring rules

The assessment used the same measurement-template structure defined in the methodology paper Smith et al. (October 2021). Rather than reproducing the full template verbatim, this appendix provides a compact excerpt summarizing the major template sections, the kinds of questions they contain, and the response formats used in the assessment.

Table B.4: Compact excerpt of the measurement template used in the assessment.

Section	Representative items and response types
Summary Information	Software name, URL, affiliation, software purpose, number of developers, initial release date, last commit date, status, licence, platform, publications, programming languages, and performance references. Response types include string, date, URL, category, and number.
Installability	Installation instructions, linearity of instructions, dependency handling, operating-system support, automation, installation breakage, validation method, uninstall support, and overall impression. Response types include yes/no, yes/no/n/a, string, number, and 1–10.
Correctness and Verifiability	Reference to requirements or theory manuals, correctness techniques, getting-started tutorial, expected output, unit tests, continuous integration, and overall impression. Response types include yes/no, yes/no/unclear, given-via-string, and 1–10.
Surface Reliability	Whether software broke during installation or tutorial testing, recoverability, descriptive error messages, and overall impression. Response types include yes/no, yes/no/n/a, and 1–10.
Surface Robustness	Handling of unexpected input and overall impression. Response types include yes/no/n/a and 1–10.
Surface Usability	Getting-started tutorial, user manual, documentation of expected user characteristics, sources of user support, and overall impression. Response types include yes/no, string, and 1–10.
Maintainability	Current version, contribution process, issue-tracking tool, issue closure percentage, code-comment percentage, version control system, and overall impression. Response types include number, yes/no, given-via-string, percentage, and 1–10.
Reusability	Number of code files, API documentation, and overall impression. Response types include number, yes/no/n/a, and 1–10.
Visibility / Transparency	Documented development process, documented development environment, release notes, and overall impression. Response types include yes/no and 1–10.
Surface Understandability	Indentation and formatting consistency, coding standard, identifier quality, comment quality, modularization, parameter naming, algorithm-name clarity, and overall impression. Response types include yes/no, yes/no/n/a, and 1–10.

Section	Representative items and response types
Raw Metrics (`git_stats`)	Text-based files, binary files, total lines, lines added, lines deleted, total commits, yearly commit counts, and monthly commit counts. Response types include number and list of numbers.
Raw Metrics (`scc`)	Text-based files, total lines, code lines, comment lines, and blank lines. Response type: number.
Repo Metrics (GitHub)	Stars, forks, watchers, open pull requests, and closed pull requests. Response type: number.

Scoring followed the conventions defined in the methodology paper Smith et al. (October 2021). Binary and categorical questions were answered using the response options specified by the template (for example, yes/no, yes/no/n/a, or yes/no/unclear). Several categories also include an overall impression item scored on a 1–10 scale. Repository-based measures were recorded as raw numeric values, and derived or normalized values were used in later comparison and ranking steps. When evidence was missing or ambiguous, responses were recorded conservatively, consistent with the approach described in Section 3.4. In the present work, evidence traceability was maintained through package-level references, specifically the project website URL and source-code repository URL recorded for each package.

Representative scoring rules

The full scoring rules follow the methodology paper Smith et al. (October 2021). Table B.5 provides representative examples of how common response types were mapped during scoring.

Category	Example scoring rule
Installability	Binary questions were typically mapped to small numeric values, for example yes/no or yes/no/n/a. The category also included an overall impression scored on a 1–10 scale.
Correctness and Verifiability	Questions such as the presence of requirements references, unit tests, or continuous integration evidence were scored using template-defined yes/no/unclear mappings, together with an overall impression on a 1–10 scale.
Surface Reliability	Breakage and recoverability questions were mapped using template-defined yes/no or yes/no/n/a scoring rules, with an overall impression on a 1–10 scale.
Surface Usability	Questions about tutorials, user manuals, and user-support artifacts were scored using yes/no or small ordinal rules, together with an overall impression on a 1–10 scale.
Maintainability	Several maintainability items used categorical or derived numeric rules, for example issue-closure percentage bands, comment-percentage bands, and identified contribution processes.
Reusability	Items such as number of code files were mapped through bucketed numeric ranges, while API documentation was scored using yes/no/n/a.
Visibility / Transparency	Questions about development process, documented environment, and release notes were scored using binary rules, with an overall impression on a 1–10 scale.
Raw repository metrics	Repository-based measures were recorded as raw numeric values (for example commits, stars, forks, lines of code, comment lines, and blank lines) and used later in normalization and aggregation.

Table B.5: Representative scoring-rule examples derived from the measurement template.

These examples are intended to illustrate the scoring approach used in the assessment; the full question set and complete scoring details are given in the methodology paper Smith et al. (October 2021).

Appendix C. AHP weights and matrices

This appendix summarizes the AHP configuration used to produce the overall ranking reported in the paper. The final ranking was generated using equal weights across the nine software-quality categories included in the comparison. The pairwise comparison interface and resulting calculated priorities are shown in Figure D.8. These materials are included to support transparency and reproducibility of the ranking configuration used in DomainX.

Category	Weight
Installability	11.1%
Correctness and Verifiability	11.1%
Surface Reliability	11.1%
Surface Robustness	11.1%
Surface Usability	11.1%
Maintainability	11.1%
Reusability	11.1%
Surface Understandability	11.1%
Visibility/Transparency	11.1%

Table C.6: Final AHP category weights used in the overall ranking.

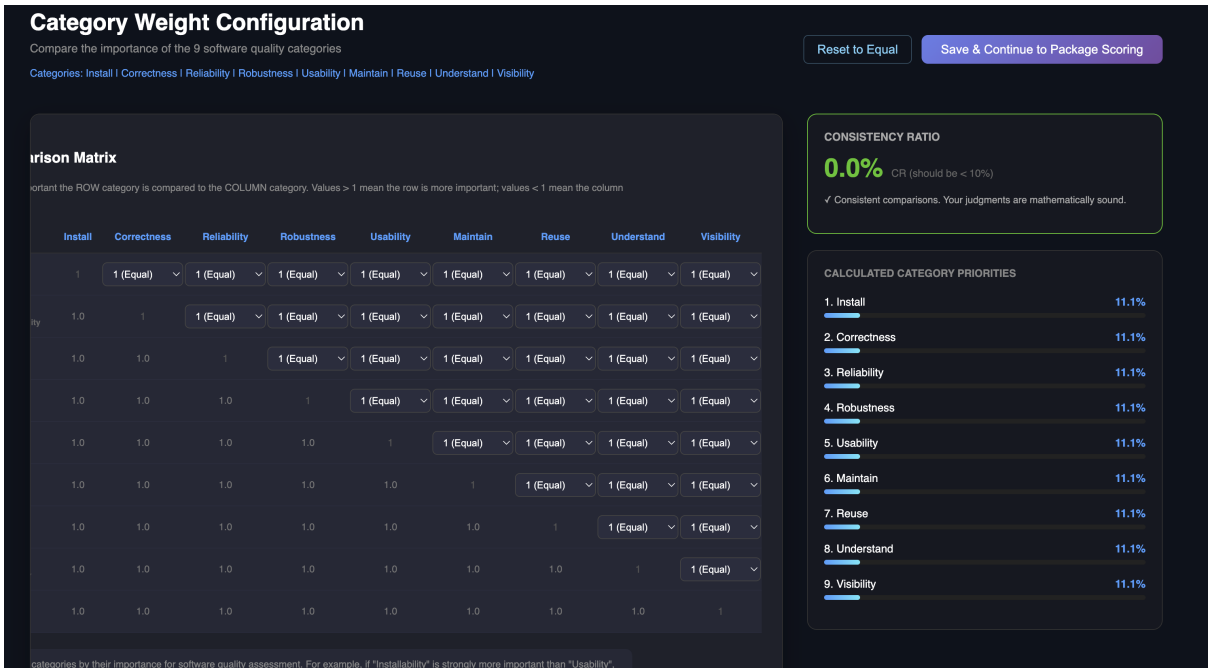


Figure C.4: DomainX AHP weight-configuration interface, including pairwise comparisons and calculated priorities.

Appendix D. DomainX screenshots and exported artifacts

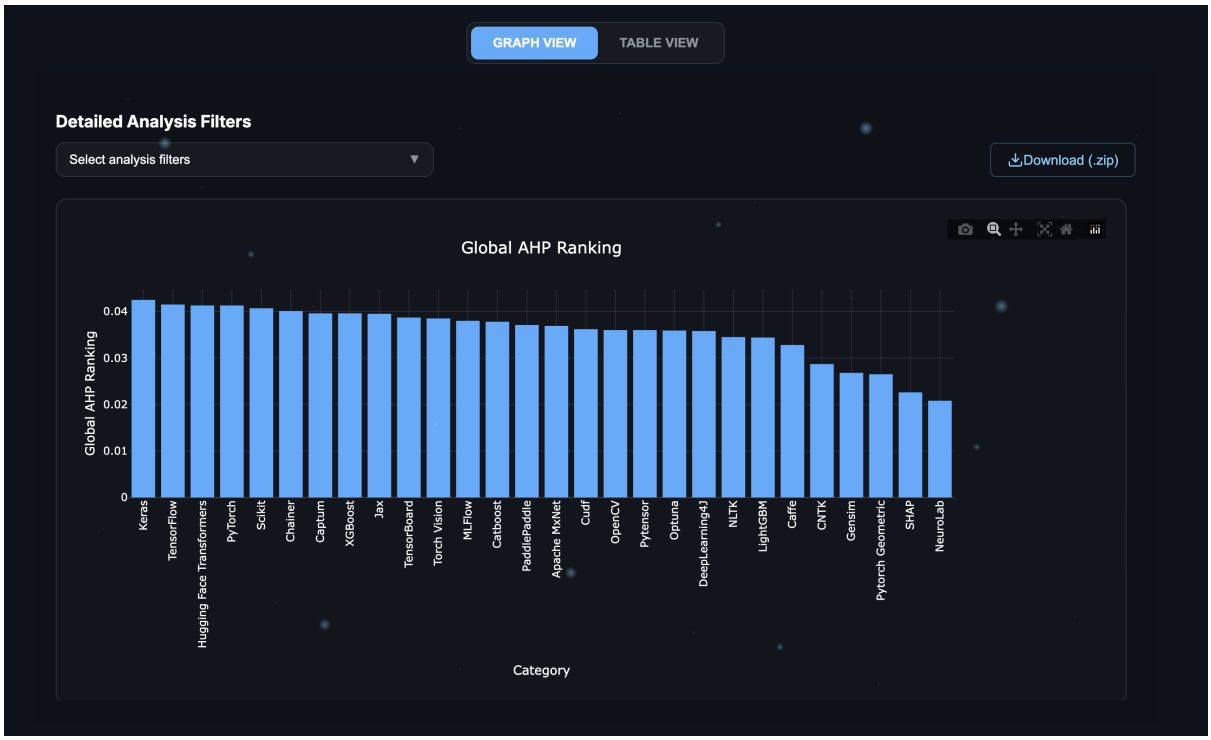
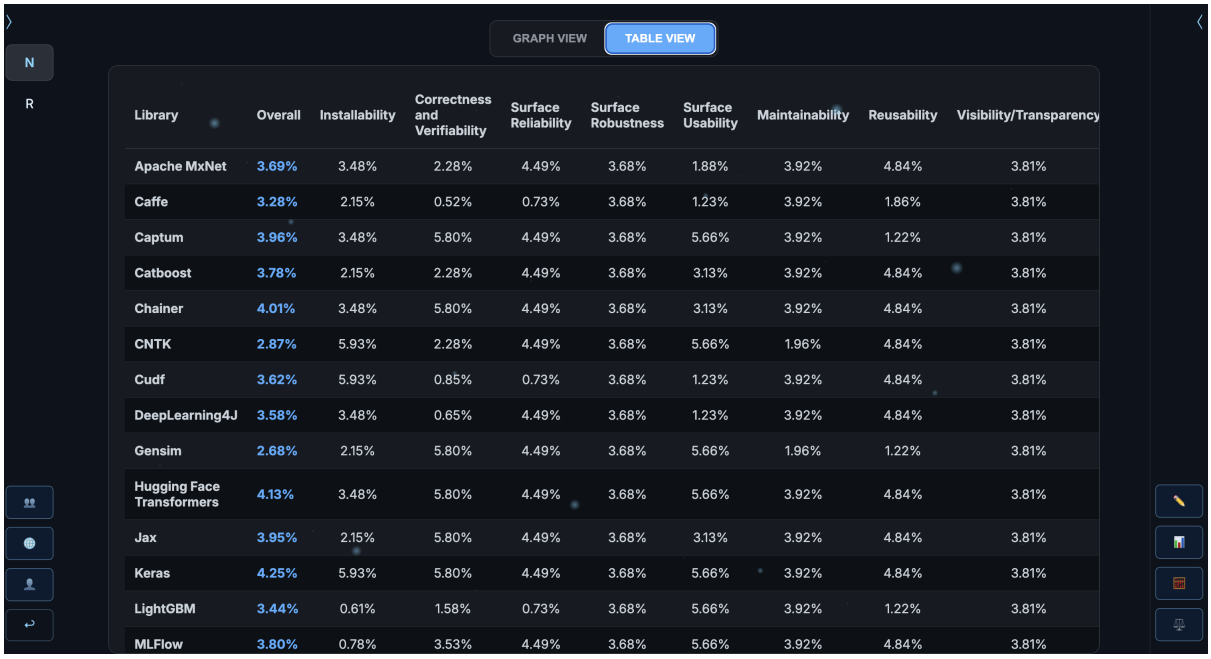


Figure D.5: DomainX graph view showing the overall AHP ranking for the shortlisted packages.



GRAPH VIEW | TABLE VIEW

Library	Overall	Installability	Correctness and Verifiability	Surface Reliability	Surface Robustness	Surface Usability	Maintainability	Reusability	Visibility/Transparency
Apache MxNet	3.69%	3.48%	2.28%	4.49%	3.68%	1.88%	3.92%	4.84%	3.81%
Caffe	3.28%	2.15%	0.52%	0.73%	3.68%	1.23%	3.92%	1.86%	3.81%
Captum	3.96%	3.48%	5.80%	4.49%	3.68%	5.66%	3.92%	1.22%	3.81%
Catboost	3.78%	2.15%	2.28%	4.49%	3.68%	3.13%	3.92%	4.84%	3.81%
Chainer	4.01%	3.48%	5.80%	4.49%	3.68%	3.13%	3.92%	4.84%	3.81%
CNTK	2.87%	5.93%	2.28%	4.49%	3.68%	5.66%	1.96%	4.84%	3.81%
Cudf	3.62%	5.93%	0.85%	0.73%	3.68%	1.23%	3.92%	4.84%	3.81%
DeepLearning4J	3.58%	3.48%	0.65%	4.49%	3.68%	1.23%	3.92%	4.84%	3.81%
Gensim	2.68%	2.15%	5.80%	4.49%	3.68%	5.66%	1.96%	1.22%	3.81%
Hugging Face Transformers	4.13%	3.48%	5.80%	4.49%	3.68%	5.66%	3.92%	4.84%	3.81%
Jax	3.95%	2.15%	5.80%	4.49%	3.68%	3.13%	3.92%	4.84%	3.81%
Keras	4.25%	5.93%	5.80%	4.49%	3.68%	5.66%	3.92%	4.84%	3.81%
LightGBM	3.44%	0.61%	1.58%	0.73%	3.68%	5.66%	3.92%	1.22%	3.81%
MLFlow	3.80%	0.78%	3.53%	4.49%	3.68%	5.66%	3.92%	4.84%	3.81%

Figure D.6: DomainX table view showing overall and per-category results for the shortlisted packages.

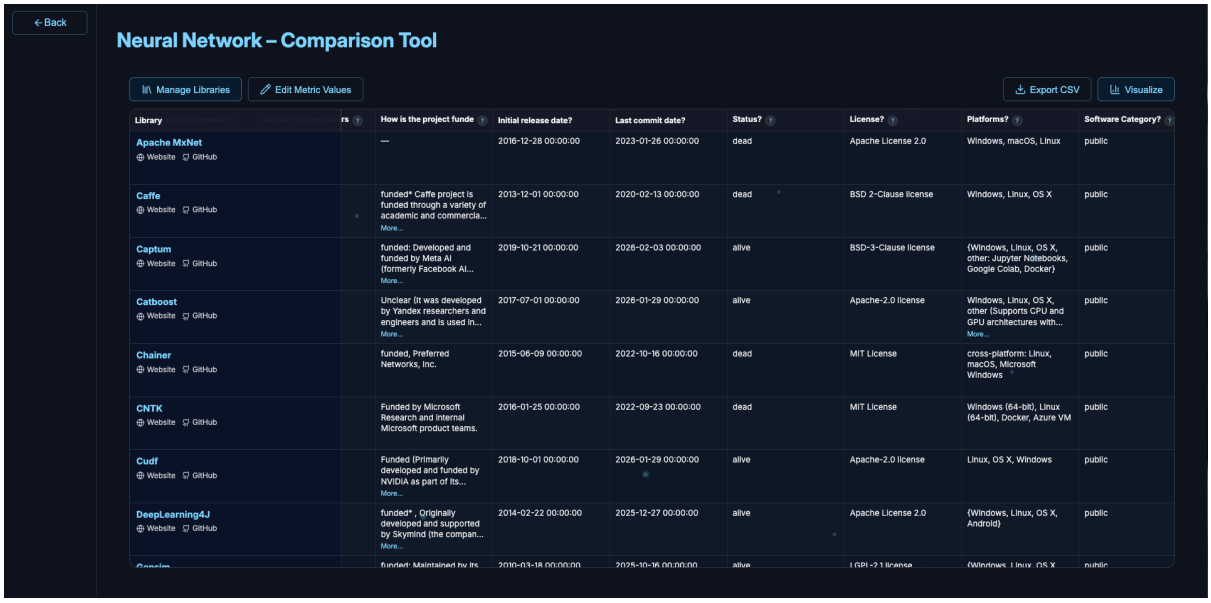


Figure D.7: DomainX comparison tool view showing package-level metadata such as website links, repository links, project status, licence, and platform information.

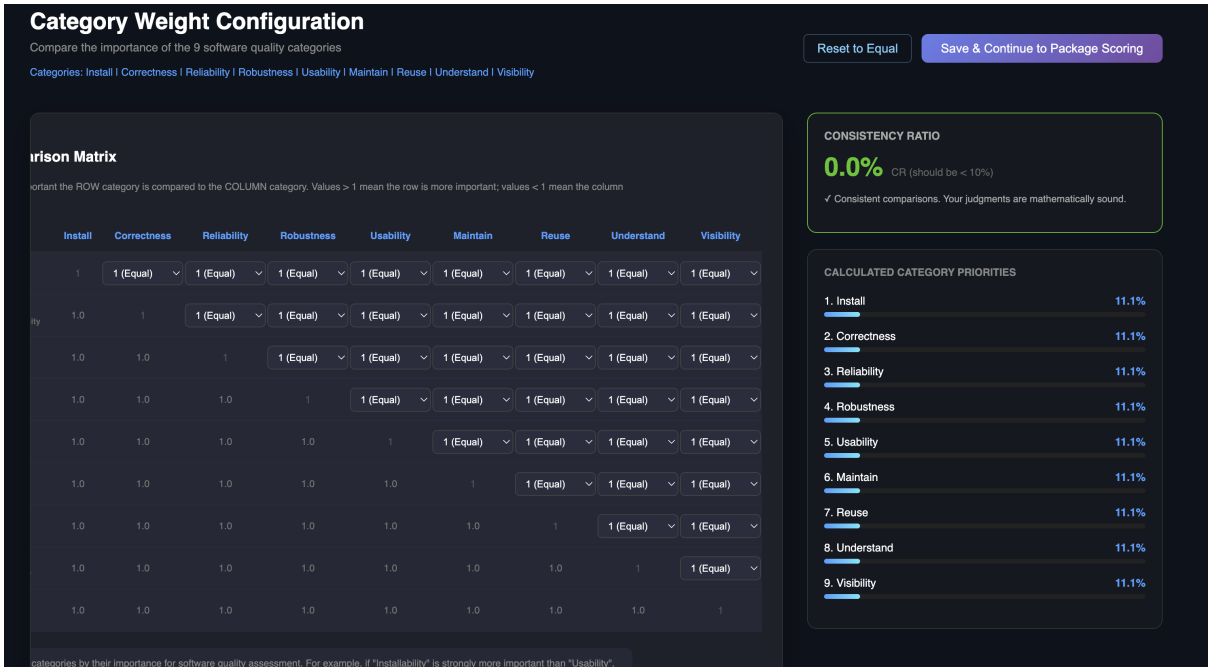


Figure D.8: DomainX weight-configuration interface used to specify category priorities for AHP-based comparison.

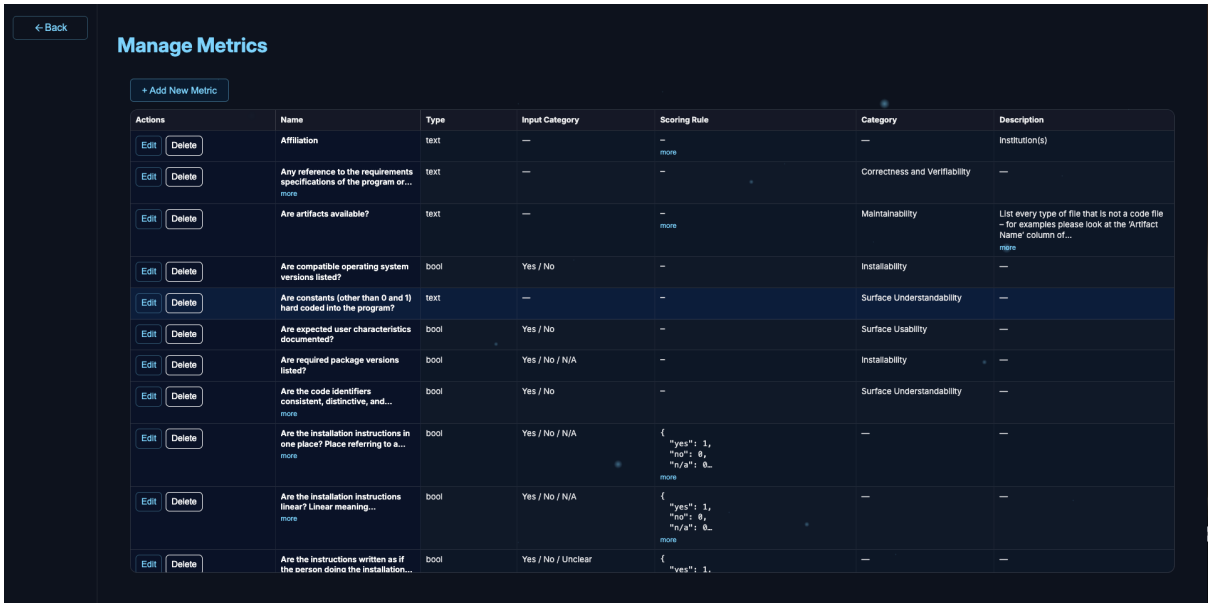


Figure D.9: DomainX metric-management interface showing how measurement items and associated metadata can be edited within the tool.

References

Spencer Smith, Jacques Carette, Peter Michalski, Ao Dong, and Olu Owojaiye. Methodology for assessing the state of the practice for domain x. *arXiv preprint arXiv:2110.11575v1*, abs/2110.11575:1–15, October 2021. URL `https://arxiv.org/abs/2110.11575`.